Fatec
Ipiranga
Pastor Enéas Tognini
.
TADS - Análise Desenvolvimento de Sistemas

Estruturas de Dados

Aula #05
Filas

Prof.: Veríssimo - carlos.pereira01@cps.sp.gov.br

10/03/2026

Sobre este documento

Este documento objetiva orientar o aluno quanto ao conteúdo a ser desenvolvido nesta disciplina, bem como a divulgação dos critérios de avaliação que serão empregados.

É muito importante que o aluno faça uma acurada leitura deste documento, pois nele conterá a descrição dos elementos que norteiam o curso.

Contents

1 Filas **1**

1.1 Introdução 1

1.2 Conceitos 1

1.2.1 Operações Básicas 2

2 Implementação **3**

2.1 Implementação usando VETORES 4

2.1.1 Implementações para Fila 5

2.2 Implementando Fila usando Ponteiros 7

2.2.1 Implementações para Fila utilizando Ponteiros 8

Chapter 1

Filas

1.1 Introdução

Em ciência da computação, as filas são uma das estruturas de dados fundamentais amplamente utilizadas para organizar e manipular elementos em uma sequência específica. No mundo, por exemplo, as pessoas aguardam sua vez em uma ordem linear, enquanto que uma fila de dados é uma coleção ordenada de elementos, onde a inserção ocorre no final da fila e a remoção ocorre no início da fila e são frequentemente usadas em uma variedade de contextos e cenários.

1.2 Conceitos

A estrutura de dados fila, como seu próprio nome já sugere, é semelhante ao funcionamento de uma fila de banco. Onde: 1) Se não há ninguém na fila e chega uma pessoa, está será o inicio e o fim da fila; 2) A partir de então, quaçquer pessoa que chegar irá para o final da fila (após a pessoa que está no fim); 3) Cada elemento a ser removido será do inicio da fila

(O primeiro que chega na fila é o primeiro que sai) conforme demonstrado à figura 1.1.

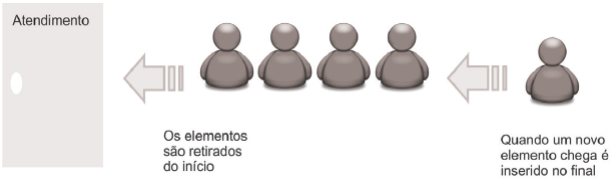

Figure 1.1: Reprentação de uma **Fila** no mundo Real

As filas são uma representação abstrata de uma coleção de elementos, onde a inserção ocorre no final da fila e a remoção ocorre no início da fila. Essa ordem de inserção e remoção é essencial para preservar a ordem dos elementos.

1.2.1 Operações Básicas

As operações básicas em uma fila são:

- Enfileirar (ou enqueuing): adicionar um elemento no final da fila.
- Desenfileirar (ou dequeuing): remover o elemento no início da fila.
- Vazio: verificar se a fila está vazia.
- Tamanho: obter o número de elementos na fila.

Chapter 2

Implementação

As possíveis formas de implementar filas são:

- Implementação com Array

  Neste caso são necessários de dois índices: um para o início da fila (front) e outro para o final da fila (rear). Ao enfileirar um elemento, incrementamos o índice rear e adicionamos o elemento nessa posição. Ao desenfileirar, removemos o elemento no índice front e incrementamos o índice front.Nesta implementação é necessário que o tamanho do array seja fixo.

- Implementação com Lista Encadeada

  Utiliza-se uma estrutura de dados encadeada para armazenar os elementos da fila. Cada elemento na fila é representado por um nó que contém o valor do elemento e um ponteiro para o próximo nó. Ao enfileirar um elemento, deve ser criado um novo nó e atualizamos os ponteiros. Ao desenfileirar, deve ser removido o primeiro nó da lista e atualizam-se os ponteiros. Essa implementação permite uma fila de tamanho dinâmico.

2.1 Implementação usando VETORES

Considerando que a operação de remoção acontece em uma extremidade e a inserção na outra, é interessante manter apontadores indicando ambos extremos da estrutura. Consequentemente, a estrutura de dados fila inclui, além do vetor que irá a comportar, as informações correspondentes, os índices correspondentes ao inicio e fim da fila, e o tamanho da fila relativo ao número de elementos contidos, conforme demonstrado à figura 2.1.

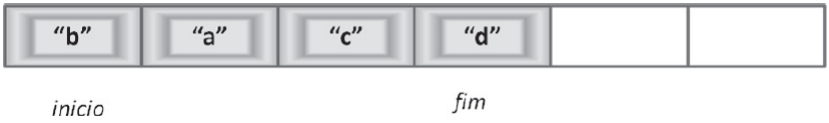

Figure 2.1: Estratégia de implementação da fila

De acordo com esta estratégia e depois da remoção de "b" e "a", e da inserção de "f" e "1", a situação da fila é representada à figura 2.2.

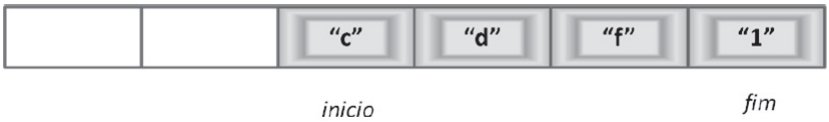

Figure 2.2: Representação da fila após a remoção

A inserção de um elemento na fila incrementa o tamanho e o índice que indica o fim da fila, enquanto que na remoção de um elemento o tamanho da fila é decrementado e o índice que indica o inicio é incrementado. Esta estratégia evita a movimentação de elementos sempre que uma inserção ou remoção for realizada. Com isso o custo de ambas operações fica constante.

2.1.1 Implementações para Fila

A listagem 2.1 mostra o funcionamento da fila.

```
/**-----------------------------------------*
 *        Implementa  es de Filas            *
  *-----------------------------------------*/

 //Estrutura da Fila
struct estrutura_Fila {
int ini;
int fim;
int quant_elementos;
tipo_base elementos [tamanho];
};
typedef struct estrutura_Fila *Fila;
 // Cria uma fila
Fila Criar(){
    Fila
    if (f = (Fila) malloc (sizeof (estrutura_Fila)
       );
        f -> ini = -1;
        f -> fim = -1;
        f -> quant_elementos = 0;
        return(f);
    }else{
        printf("Nao existe memoria suficiente.");
        return;
    }
};
//Inserir na Fila
void inserir (Fila f, tipo_base v) {
    if (f -> quant_elementos == tamanho) {
        printf (  Fila   cheia!   );
        return;
    }
        f -> fim = incrementar (f -> fim);
            if(f-> quant_elementos == 0){
        }
            f -> elementos [f-> fim] = v;
            f -> quant_elementos++;
}
```

```
//Remover da Fila
int remover(Fila f){
    if (f -> quant_elementos == 0);
    printf("Fila vazia.");
    return;
    }
    int temp = f -> elementos[f -> ini];
    f -> ini = incrementar(f -> ini);
    (f -> quant_elementos)--;
    if(f -> quant_elementos == 0){;
        f -> ini = -1;
        f -> fim = -1;
    }
    return(temp);
}
```

Listing 2.1: funcionamento da Fila

2.2 Implementando Fila usando Ponteiros

A implementação de filas utilizando ponteiros segue a mesma estratégia analisada na seção anterior. Como teremos que inserir e retirar elementos das extremidades opostas da lista, representando o início e o fim da fila, é preciso utilizar dois ponteiros, ini e fim, que apontam respectivamente para o primeiro e para o último nó da fila. A partir desta necessidade se faz indispensável a implementação da fila utilizando um nó diferenciado, chamado de header ou cabeçalho, para conter estes ponteiros, conforme demonstrado à figura 2.3.

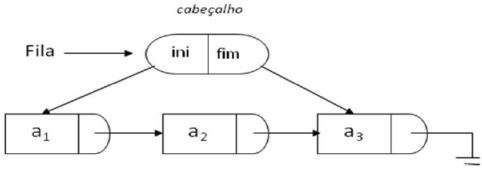

Figure 2.3: Representação da fila utilizando ponteiros

2.2.1 Implementações para Fila utilizando Ponteiros

A listagem 2.2 mostra o funcionamento da fila, com a utilização de ponteiros.

```
/*-----------------------------------------------*/
/*          FATEC-Ipiranga                       */
/*      ADS - Estrutura de Dados                 */
/*       Id da Atividade: N1-1                   */
/*Objetivo: Demonstrar manipula  o de fila      */
/*          com ponteiro                         */
/*                                               */
/*      Autor: Prof. Verissimo                  */
/*          Data:10/03/2026                      */
/*-----------------------------------------------*/
#include <stdio.h>
#include <stdlib.h>

// Defini  o do tipo de dado
typedef int tipo_base;

// Estrutura do N
typedef struct no {
    tipo_base elemento;
    struct no *prox;
} no;
typedef struct no *no_ptr;

// Estrutura de Controle
struct cabecalho {
    no_ptr ini;
    no_ptr fim;
};
typedef struct cabecalho *Fila;

// --- Fun  es Principais ---

Fila Criar() {
    Fila f = (Fila) malloc(sizeof(struct cabecalho
        ));
```

```
35     if (f != NULL) {
36         f->ini = NULL;
37         f->fim = NULL;
38     }
39     return f;
40 }
41
42 int Vazia(Fila f) {
43     return (f == NULL || f->ini == NULL);
44 }
45
46 int inserir(Fila f, tipo_base dado) {
47     no_ptr novo = (no_ptr) malloc(sizeof(no));
48     if (novo == NULL) return 0;
49     novo->elemento = dado;
50     novo->prox = NULL;
51
52     if (f->ini == NULL) {
53         f->ini = novo;
54     } else {
55         f->fim->prox = novo;
56     }
57     f->fim = novo;
58     return 1;
59 }
60
61 tipo_base remover(Fila f) {
62     if (Vazia(f)) return -1;
63     no_ptr temp = f->ini;
64     tipo_base valor = temp->elemento;
65     f->ini = f->ini->prox;
66     if (f->ini == NULL) f->fim = NULL;
67     free(temp);
68     return valor;
69 }
70
71 // --- Nova Fun   o: Listar Fila ---
72 void listarFila(Fila f) {
73     // Uso da fun   o Vazia para tratar a
           exce   o de fila sem elementos
74     if (Vazia(f)) {
```

```
        printf("\n[AVISO] A fila esta vazia. Nao ha nada para listar.\n");
        return;
    }

    no_ptr atual = f->ini;
    printf("\nElementos na fila: ");
    while (atual != NULL) {
        printf("[%d] ", atual->elemento);
        atual = atual->prox;
    }
    printf("\n");
}

// --- Menu e Main ---

int main() {
    Fila f = Criar();
    int opcao;
    tipo_base valor;

    if (f == NULL) {
        printf("Erro de memoria.\n");
        return 1;
    }

    do {
        printf("\n1-Inserir na fila\n2-Remover da Fila\n3-lista fila\n0-Sair\nEscolha: ");
        if (scanf("%d", &opcao) != 1) {
            while (getchar() != '\n'); // Limpa buffer se digitarem letra
            opcao = -1;
        }

        switch (opcao) {
            case 1:
                printf("Valor para inserir: ");
                scanf("%d", &valor);
                if (inserir(f, valor)) printf("Sucesso.\n");
```

```
112                 break;
113
114             case 2:
115                 if (Vazia(f)) {
116                     printf("Erro: Fila Vazia.\n");
117                 } else {
118                     printf("Valor %d removido.\n",
                            remover(f));
119                 }
120                 break;
121
122             case 3:
123                 listarFila(f);
124                 break;
125
126             case 0:
127                 printf("Saindo...\n");
128                 break;
129
130             default:
131                 printf("Opcao invalida.\n");
132                 break;
133         }
134     } while (opcao != 0);
135
136     // Liberar mem ria final
137     while (!Vazia(f)) remover(f);
138     free(f);
139
140     return 0;
141 }
```

Listing 2.2: funcionamento da Fila utilizando Ponteiros